

工具系统与 Hook 体系

工具系统

工具注册

源文件: `src/plugin/tool-registry.ts` (133 行) — `createToolRegistry()` 工厂函数

```
// src/plugin/tool-registry.ts - 完整源码
export function createToolRegistry(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  managers: Pick<Managers, "backgroundManager" | "tmuxSessionManager" | "skillMcpManager">
  skillContext: SkillContext
  availableCategories: AvailableCategory[]
}): ToolRegistryResult {
  const { ctx, pluginConfig, managers, skillContext, availableCategories } = args

  const backgroundTools = createBackgroundTools(managers.backgroundManager, ctx.client)
  const callOmoAgent = createCallOmoAgent(ctx, managers.backgroundManager, pluginConfig.disabled_agents ?? [])

  const isMultimodalLookerEnabled = !(pluginConfig.disabled_agents ?? []).some(
    (agent) => agent.toLowerCase() === "multimodal-looker",
  )
  const lookAt = isMultimodalLookerEnabled ? createLookAt(ctx) : null

  const delegateTask = createDelegateTask({
    manager: managers.backgroundManager,
    client: ctx.client,
    directory: ctx.directory,
    userCategories: pluginConfig.categories,
    agentOverrides: pluginConfig.agents,
    gitMasterConfig: pluginConfig.git_master,
    sisyphusJuniorModel: pluginConfig.agents?.["sisyphus-junior"]?.model,
    browserProvider: skillContext.browserProvider,
    disabledSkills: skillContext.disabledSkills,
    availableCategories,
    availableSkills: skillContext.availableSkills,
    onSyncSessionCreated: async (event) => {
      await managers.tmuxSessionManager.onSessionCreated({
        type: "session.created",
        properties: { info: { id: event.sessionID, parentID: event.parentID, title: event.title } },
      })
    },
  })

  const skillMcpTool = createSkillMcpTool({ manager: managers.skillMcpManager, /* ... */ })
  const skillTool = createSkillTool({ commands, skills: skillContext.mergedSkills, /* ... */ })

  const taskSystemEnabled = pluginConfig.experimental?.task_system ?? false
  const taskToolsRecord: Record<string, ToolDefinition> = taskSystemEnabled
    ? { task_create: createTaskCreateTool(pluginConfig, ctx), task_get: createTaskGetTool(pluginConfig),
      task_list: createTaskList(pluginConfig), task_update: createTaskUpdateTool(pluginConfig, ctx) }
    : {}

  const hashlineEnabled = pluginConfig.hashline_edit ?? true
  const hashlineToolsRecord = hashlineEnabled ? { edit: createHashlineEditTool() } : {}

  const allTools: Record<string, ToolDefinition> = {
    ...builtinTools, // 内置工具
    ...createGrepTools(ctx), // grep 文本搜索
    ...createGlobTools(ctx), // glob 文件匹配
    ...createAstGrepTools(ctx), // AST 搜索
    ...createSessionManagerTools(ctx), // 会话管理
    ...backgroundTools, // background_output, background_cancel
    call_omo_agent: callOmoAgent, // 直接调用 Agent
    ...(lookAt ? { look_at: lookAt } : {}), // 多模态查看
  }
}
```

```
task: delegateTask,                // 核心委派工具
skill_mcp: skillMcpTool,           // 技能 MCP
skill: skillTool,                  // 技能执行
interactive_bash,                  // 交互式 Shell
...taskToolsRecord,               // task_create/get/list/update (实验性)
...hashlineToolsRecord,           // hashline edit
}

const filteredTools = filterDisabledTools(allTools, pluginConfig.disabled_tools)
return { filteredTools, taskSystemEnabled }
}
```

26 个工具详解

搜索类

工具	源文件	功能
grep	tools/grep/	正则/文本搜索工作区文件
glob	tools/glob/	文件名模式匹配
ast_grep	tools/ast-grep/	基于 AST 的结构化代码搜索（依赖 @ast-grep/napi）

编排类

工具	源文件	功能
task	tools/delegate-task/	核心委派工具 — 分类/Agent 双路径任务分发
background_output	tools/background-task/	获取后台任务结果
background_cancel	tools/background-task/	取消后台任务
call_omo_agent	tools/call-omo-agent/	直接调用命名 Agent

技能类

工具	源文件	功能
skill	tools/skill/	执行 SKILL.md 技能
skill_mcp	tools/skill-mcp/	调用技能内嵌的 MCP 服务

LSP 类

工具	源文件	功能
lsp_diagnostics	tools/lsp/	获取文件诊断信息
lsp_hover	tools/lsp/	悬停信息
lsp_find_references	tools/lsp/	查找引用
lsp_go_to_definition	tools/lsp/	跳转定义
lsp_rename	tools/lsp/	重命名符号

会话类

工具	源文件	功能
session_*	tools/session-manager/	会话创建/列表/续传管理

编辑类

工具	源文件	功能
hashline_edit	tools/hashline-edit/	基于行号 hash 的精确编辑

其他

工具	源文件	功能
look_at	tools/look-at/	多模态内容查看（图片、截图）
interactive_bash	tools/interactive-bash/	交互式终端
slashcommand	tools/slashcommand/	斜杠命令执行

任务 CRUD（实验性）

工具	源文件	功能
TaskCreate	tools/task/task-create.ts	创建任务
TaskList	tools/task/task-list.ts	列出任务
TaskGet	tools/task/task-get.ts	获取任务详情
TaskUpdate	tools/task/task-update.ts	更新任务状态

工具创建工厂模式

所有工具遵循 tool() 工厂模式：

```
import { tool, type ToolDefinition } from "@opencode-ai/plugin"

function createXXXTool(options: XXXToolOptions): ToolDefinition {
  return tool({
    description: "工具描述",
    args: {
      param1: tool.schema.string().describe("参数描述"),
      param2: tool.schema.boolean().describe("参数描述"),
    },
    async execute(args, toolContext) {
      // 执行逻辑
      return "结果字符串"
    },
  })
}
```

Hook 体系

三层架构

源文件: src/plugin/hooks/create-core-hooks.ts (51 行) — 聚合入口

```
// src/plugin/hooks/create-core-hooks.ts — 完整源码
export function createCoreHooks(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  modelCacheState: ModelCacheState
  isHookEnabled: (hookName: HookName) => boolean
  safeHookEnabled: boolean
}) {
  const { ctx, pluginConfig, modelCacheState, isHookEnabled, safeHookEnabled } = args

  // 第一层: 会话生命周期 (23 个 Hook)
  const session = createSessionHooks({
    ctx, pluginConfig, modelCacheState, isHookEnabled, safeHookEnabled,
  })

  // 第二层: 工具守卫 (10 个 Hook)
  const tool = createToolGuardHooks({
    ctx, pluginConfig, modelCacheState, isHookEnabled, safeHookEnabled,
  })

  // 第三层: 消息变换 (4 个 Hook)
  const transform = createTransformHooks({
    ctx, pluginConfig, isHookEnabled: (name) => isHookEnabled(name as HookName), safeHookEnabled,
  })

  return { ...session, ...tool, ...transform } // 扁平合并为 37 个 hook
}
```

通用 Hook 创建模式 — 所有 Hook 都使用双重守卫:

```
// 模式: isHookEnabled() + safeCreateHook() 双重守卫
const hook = isHookEnabled("hook-name")
  ? safeCreateHook("hook-name", () => createXXXHook(ctx, options), { enabled: safeHookEnabled })
  : null
```

Hook 启用/禁用

```
// src/index.ts — Hook 启用控制
const disabledHooks = new Set(pluginConfig.disabled_hooks ?? [])
const isHookEnabled = (hookName: HookName): boolean => !disabledHooks.has(hookName)
const safeHookEnabled = pluginConfig.experimental?.safe_hook_creation ?? true
```

safeHookEnabled 控制是否在 Hook 创建失败时优雅降级 (默认开启)。

会话生命周期 Hooks (23 个)

源文件: src/plugin/hooks/create-session-hooks.ts (290 行)

```
// src/plugin/hooks/create-session-hooks.ts - SessionHooks 类型定义 (23 个可空字段)
export type SessionHooks = {
  contextWindowMonitor: ReturnType<typeof createContextWindowMonitorHook> | null
  preemptiveCompaction: ReturnType<typeof createPreemptiveCompactionHook> | null
  sessionRecovery: ReturnType<typeof createSessionRecoveryHook> | null
  sessionNotification: ReturnType<typeof createSessionNotification> | null
  thinkMode: ReturnType<typeof createThinkModeHook> | null
  modelFallback: ReturnType<typeof createModelFallbackHook> | null
  anthropicContextWindowLimitRecovery: ReturnType<typeof createAnthropicContextWindowLimitRecoveryHook> | null
  autoUpdateChecker: ReturnType<typeof createAutoUpdateCheckerHook> | null
  agentUsageReminder: ReturnType<typeof createAgentUsageReminderHook> | null
  nonInteractiveEnv: ReturnType<typeof createNonInteractiveEnvHook> | null
  interactiveBashSession: ReturnType<typeof createInteractiveBashSessionHook> | null
  ralphLoop: ReturnType<typeof createRalphLoopHook> | null
  editErrorRecovery: ReturnType<typeof createEditErrorRecoveryHook> | null
  delegateTaskRetry: ReturnType<typeof createDelegateTaskRetryHook> | null
  startWork: ReturnType<typeof createStartWorkHook> | null
  prometheusMdOnly: ReturnType<typeof createPrometheusMdOnlyHook> | null
  sisyphusJuniorNotepad: ReturnType<typeof createSisyphusJuniorNotepadHook> | null
  noSisyphusGpt: ReturnType<typeof createNoSisyphusGptHook> | null
  noHephaestusNonGpt: ReturnType<typeof createNoHephaestusNonGptHook> | null
  questionLabelTruncator: ReturnType<typeof createQuestionLabelTruncatorHook> | null
  taskResumeInfo: ReturnType<typeof createTaskResumeInfoHook> | null
  anthropicEffort: ReturnType<typeof createAnthropicEffortHook> | null
  runtimeFallback: ReturnType<typeof createRuntimeFallbackHook> | null
}
```

Hook	源目录	功能	触发时机
context-window-monitor	hooks/context-window-monitor/	监控上下文窗口使用率	chat.message
preemptive-compaction	hooks/preemptive-compaction/	预防性压缩（实验性）	chat.message
session-recovery	hooks/session-recovery/	会话崩溃恢复	event (error)
session-notification	hooks/session-notification/	桌面通知（支持声音）	event (idle)
think-mode	hooks/think-mode/	Extended thinking 控制	chat.message
model-fallback	hooks/model-fallback/	模型错误自动切换 + 标题更新	event (error)
anthropic-context-window-limit-recovery	hooks/anthropic-*--recovery/	Anthropic 上下文限制恢复	event (error)
anthropic-effort	hooks/anthropic-effort/	Anthropic effort level 调整	chat.params
auto-update-checker	hooks/auto-update-checker/	自动更新检测	event (created)
ralph-loop	hooks/ralph-loop/	Ralph 循环模式	chat.message
prometheus-md-only	hooks/prometheus-md-only/	Prometheus 仅 .md 写入	tool.execute.before
start-work	hooks/start-work/	/start-work 命令	chat.message
no-sisyphus-gpt	hooks/no-sisyphus-gpt/	阻止 Sisyphus 用 GPT	event

Hook	源目录	功能	触发时机
no-hephaestus-non-gpt	hooks/no-hephaestus-non-gpt/	阻止 Hephaestus 用非 GPT	event
runtime-fallback	hooks/runtime-fallback/	运行时错误 fallback	event (error)
non-interactive-env	hooks/non-interactive-env/	非交互环境检测	event
question-label-truncator	hooks/question-label-truncator/	问题标签截断	tool.execute.before
agent-usage-reminder	hooks/agent-usage-reminder/	Agent 使用提醒	chat.message
sisyphus-junior-notepad	hooks/sisyphus-junior-notepad/	SJ 笔记本	tool.execute.after
session-todo-status	hooks/session-todo-status/	会话 Todo 状态	event
delegate-task-retry	hooks/delegate-task-retry/	task() 失败自动重试	tool.execute.after
task-reminder	hooks/task-reminder/	任务提醒	chat.message
task-resume-info	hooks/task-resume-info/	任务恢复信息	chat.message

工具守卫 Hooks (10 个)

源文件: src/plugin/hooks/create-tool-guard-hooks.ts (121 行)

```
// src/plugin/hooks/create-tool-guard-hooks.ts - ToolGuardHooks 类型
export type ToolGuardHooks = {
  commentChecker: ReturnType<typeof createCommentCheckerHooks> | null
  toolOutputTruncator: ReturnType<typeof createToolOutputTruncatorHook> | null
  directoryAgentsInjector: ReturnType<typeof createDirectoryAgentsInjectorHook> | null
  directoryReadmeInjector: ReturnType<typeof createDirectoryReadmeInjectorHook> | null
  emptyTaskResponseDetector: ReturnType<typeof createEmptyTaskResponseDetectorHook> | null
  rulesInjector: ReturnType<typeof createRulesInjectorHook> | null
  tasksTodowriteDisabler: ReturnType<typeof createTasksTodowriteDisablerHook> | null
  writeExistingFileGuard: ReturnType<typeof createWriteExistingFileGuardHook> | null
  hashlineReadEnhancer: ReturnType<typeof createHashlineReadEnhancerHook> | null
  jsonErrorRecovery: ReturnType<typeof createJsonErrorRecoveryHook> | null
}
```

directory-agents-injector 版本自动禁用逻辑 — 当 OpenCode 原生支持 Agent 注入时自动跳过:

```
// 特殊逻辑: OpenCode 版本检测 → 自动禁用
let directoryAgentsInjector: ReturnType<typeof createDirectoryAgentsInjectorHook> | null = null
if (isHookEnabled("directory-agents-injector")) {
  const currentVersion = getOpenCodeVersion()
  const hasNativeSupport =
    currentVersion !== null && isOpenCodeVersionAtLeast(OPENCODE_NATIVE_AGENTS_INJECTION_VERSION)
  if (hasNativeSupport) {
    log("directory-agents-injector auto-disabled due to native OpenCode support", {
      currentVersion, nativeVersion: OPENCODE_NATIVE_AGENTS_INJECTION_VERSION,
    })
  } else {
    directoryAgentsInjector = safeHook("directory-agents-injector", () =>
      createDirectoryAgentsInjectorHook(ctx, modelCacheState))
  }
}
```

Hook	源目录	功能	触发时机
comment-checker	hooks/comment-checker/	AI 生成注释检测（强制执行）	tool.execute.after
tool-output-truncator	hooks/tool-output-truncator/	超长输出截断	tool.execute.after
directory-agents-injector	hooks/directory-agents-injector/	目录级 Agent 注入	tool.execute.before
directory-readme-injector	hooks/directory-readme-injector/	README 注入	tool.execute.before
rules-injector	hooks/rules-injector/	规则文件注入	tool.execute.before
write-existing-file-guard	hooks/write-existing-file-guard/	写入已有文件保护	tool.execute.before
hashline-read-enhancer	hooks/hashline-read-enhancer/	Hashline 读取增强	tool.execute.after
json-error-recovery	hooks/json-error-recovery/	JSON 解析错误恢复	tool.execute.after
tasks-todowrite-disabler	hooks/tasks-todowrite-disabler/	Task 系统中禁用 TodoWrite	tool.execute.before
empty-task-response-detector	hooks/empty-task-*--detector/	空任务响应检测	tool.execute.after

消息变换 Hooks (4 个)

源文件: src/plugin/hooks/create-transform-hooks.ts (75 行, 完整源码)

```
// src/plugin/hooks/create-transform-hooks.ts - TransformHooks 类型
export type TransformHooks = {
  claudeCodeHooks: ReturnType<typeof createClaudeCodeHooksHook> | null
  keywordDetector: ReturnType<typeof createKeywordDetectorHook> | null
  contextInjectorMessagesTransform: ReturnType<typeof createContextInjectorMessagesTransformHook>
  // ↑ 注意: 此字段非可空! contextInjector 始终启用
  thinkingBlockValidator: ReturnType<typeof createThinkingBlockValidatorHook> | null
}

// contextInjectorMessagesTransform 始终创建 (不受 isHookEnabled 控制)
const contextInjectorMessagesTransform =
  createContextInjectorMessagesTransformHook(contextCollector) // 使用单例 contextCollector
```


Hook	源目录	功能	触发时机
claude-code-hooks	hooks/claude-code-hooks/	兼容 Claude Code hooks 执行	messages.transform
keyword-detector	hooks/keyword-detector/	关键词检测 (触发上下文注入)	messages.transform
context-injector-messages-transform	features/context-injector/	上下文注入	messages.transform
thinking-block-validator	hooks/thinking-block-validator/	思考块验证 (修复格式错误)	messages.transform

延续 Hooks (7 个)

源文件: src/plugin/hooks/create-continuation-hooks.ts (127 行)

```
// src/plugin/hooks/create-continuation-hooks.ts - 核心类型和工厂
export type ContinuationHooks = {
  stopContinuationGuard: ReturnType<typeof createStopContinuationGuardHook> | null
  compactionContextInjector: ReturnType<typeof createCompactionContextInjector> | null
  compactionTodoPreserver: ReturnType<typeof createCompactionTodoPreserverHook> | null
  todoContinuationEnforcer: ReturnType<typeof createTodoContinuationEnforcer> | null
  unstableAgentBabysitter: ReturnType<typeof createUnstableAgentBabysitter> | null
  backgroundNotificationHook: ReturnType<typeof createBackgroundNotificationHook> | null
  atlasHook: ReturnType<typeof createAtlasHook> | null
}

export function createContinuationHooks(args: {
  ctx: PluginContext
  pluginConfig: OhMyOpenCodeConfig
  isHookEnabled: (hookName: HookName) => boolean
  safeHookEnabled: boolean
  backgroundManager: BackgroundManager // 额外依赖: 后台管理器
  sessionRecovery: SessionRecovery // 额外依赖: 会话恢复
}): ContinuationHooks {
  // todoContinuationEnforcer 依赖 stopContinuationGuard?.isStopped (hook 间依赖)
  const todoContinuationEnforcer = isHookEnabled("todo-continuation-enforcer")
    ? safeHook("todo-continuation-enforcer", () =>
      createTodoContinuationEnforcer(ctx, {
        backgroundManager,
        isContinuationStopped: stopContinuationGuard?.isStopped,
      })
    )
    : null
  // ...
}
```

Hook	源目录	功能	触发时机
stop-continuation-guard	hooks/stop-continuation-guard/	停止延续条件守卫	chat.message
compaction-context-injector	hooks/compaction-context-injector/	压缩后上下文注入	session.compacting
compaction-todo-preserver	hooks/compaction-todo-preserver/	压缩时保留 Todo 状态	session.compacting
todo-continuation-enforcer	hooks/todo-continuation-enforcer/	Todo 未完成强制继续	chat.message

Hook	源目录	功能	触发时机
unstable-agent-babysitter	hooks/unstable-agent-babysitter/	不稳定模型崩溃恢复	event (error)
background-notification	hooks/background-notification/	后台任务完成通知注入	chat.message
atlas	hooks/atlas/	Atlas 编排器 hook	chat.message

技能 Hooks (2 个)

源文件: src/plugin/hooks/create-skill-hooks.ts (41 行, 完整源码)

```
// src/plugin/hooks/create-skill-hooks.ts - 完整源码
export type SkillHooks = {
  categorySkillReminder: ReturnType<typeof createCategorySkillReminderHook> | null
  autoSlashCommand: ReturnType<typeof createAutoSlashCommandHook> | null
}

export function createSkillHooks(args: {
  ctx: PluginContext
  isHookEnabled: (hookName: HookName) => boolean
  safeHookEnabled: boolean
  mergedSkills: LoadedSkill[]
  availableSkills: AvailableSkill[]
}): SkillHooks {
  const { ctx, isHookEnabled, safeHookEnabled, mergedSkills, availableSkills } = args

  const safeHook = <T>(hookName: HookName, factory: () => T): T | null =>
    safeCreateHook(hookName, factory, { enabled: safeHookEnabled })

  const categorySkillReminder = isHookEnabled("category-skill-reminder")
    ? safeHook("category-skill-reminder", () =>
      createCategorySkillReminderHook(ctx, availableSkills))
    : null

  const autoSlashCommand = isHookEnabled("auto-slash-command")
    ? safeHook("auto-slash-command", () =>
      createAutoSlashCommandHook({ skills: mergedSkills }))
    : null

  return { categorySkillReminder, autoSlashCommand }
}
```

Hook	源目录	功能	触发时机
category-skill-reminder	hooks/category-skill-reminder/	提醒可用技能	chat.message
auto-slash-command	hooks/auto-slash-command/	自动匹配斜杠命令	chat.message

Hook 在 OpenCode 生命周期中的分布

chat.message

- └─ context-window-monitor
- └─ preemptive-compaction
- └─ think-mode
- └─ ralph-loop
- └─ start-work
- └─ agent-usage-reminder
- └─ task-reminder / task-resume-info
- └─ stop-continuation-guard
- └─ todo-continuation-enforcer
- └─ background-notification
- └─ atlas
- └─ category-skill-reminder
- └─ auto-slash-command

chat.params

- └─ anthropic-effort

event (session.created / deleted / idle / error)

- └─ session-recovery (error)
- └─ session-notification (idle)
- └─ model-fallback (error)
- └─ anthropic-context-window-limit-recovery (error)
- └─ auto-update-checker (created)
- └─ no-sisyphus-gpt / no-hephaestus-non-gpt
- └─ runtime-fallback (error)
- └─ non-interactive-env
- └─ session-todo-status
- └─ unstable-agent-babysitter (error)

tool.execute.before

- └─ prometheus-md-only
- └─ directory-agents-injector
- └─ directory-readme-injector
- └─ rules-injector
- └─ write-existing-file-guard
- └─ tasks-todowrite-disabler
- └─ question-label-truncator

tool.execute.after

- └─ comment-checker
- └─ tool-output-truncator
- └─ hashline-read-enhancer
- └─ json-error-recovery
- └─ empty-task-response-detector
- └─ delegate-task-retry
- └─ sisyphus-junior-notepad

experimental.chat.messages.transform

- └─ claude-code-hooks
- └─ keyword-detector
- └─ context-injector-messages-transform
- └─ thinking-block-validator

experimental.session.compacting

```
├─ compaction-context-injector
└─ compaction-todo-preserver
```

关键 Hook 深入

comment-checker — AI 注释检测

拦截 Agent 生成的代码中常见的 AI 腔调注释（如 "// Handle edge case elegantly"），强制移除或报错。

todo-continuation-enforcer — Todo 强制延续

当 Agent 的 Todo 列表有未完成项时，阻止 Agent 提前结束，注入提醒消息强制继续工作。

unstable-agent-babysitter — 不稳定模型保姆

监控 Gemini/MiniMax 等模型的错误率，在崩溃时自动恢复或切换到稳定模型。

model-fallback — 模型自动切换

当模型 API 返回错误时，自动切换到 fallback 链中的下一个模型，并更新会话标题。

prometheus-md-only — Prometheus 文件限制

拦截 Prometheus Agent 的所有写操作，确保只能写入 `.sisyphus/plans/` 和 `.sisyphus/drafts/` 目录。

rules-injector — 规则注入

在工具执行前，注入项目级规则文件（`.opencode/rules/`），确保 Agent 行为符合项目约定。

Hook 目录完整索引

```
src/hooks/
├─ agent-usage-reminder/
├─ anthropic-context-window-limit-recovery/
├─ anthropic-effort/
├─ atlas/
├─ auto-slash-command/
├─ auto-update-checker/
├─ background-notification/
├─ category-skill-reminder/
├─ claude-code-hooks/
├─ comment-checker/
├─ compaction-context-injector/
├─ compaction-todo-presenter/
├─ context-window-monitor/
├─ delegate-task-retry/
├─ directory-agents-injector/
├─ directory-readme-injector/
├─ edit-error-recovery/
├─ empty-task-response-detector/
├─ hashline-edit-diff-enhancer/
├─ hashline-read-enhancer/
├─ json-error-recovery/
├─ keyword-detector/
├─ model-fallback/
├─ no-hephaestus-non-gpt/
├─ no-sisyphus-gpt/
├─ non-interactive-env/
├─ preemptive-compaction/
├─ prometheus-md-only/
├─ question-label-truncator/
├─ ralph-loop/
├─ rules-injector/
├─ runtime-fallback/
├─ session-notification/
├─ session-recovery/
├─ session-todo-status/
├─ sisyphus-junior-notepad/
├─ start-work/
├─ stop-continuation-guard/
├─ task-reminder/
├─ task-resume-info/
├─ tasks-todowrite-disabler/
├─ think-mode/
├─ thinking-block-validator/
├─ todo-continuation-enforcer/
├─ tool-output-truncator/
├─ unstable-agent-babysitter/
├─ write-existing-file-guard/
```